Frequency Evaluation: Mandatory Work Assignment #1

3S – Secure Streaming Solution: PART I

Design, Implementation, and Experimental Evaluation of a Security Solution for Real-Time Media Streaming using Flexible Cryptographic Configurations

Summary

In this assignment, students will develop **Part I of a project assignment** involving the design, implementation, and experimental evaluation of a secure, real-time media-streaming system. The complete system comprises three components: an Authentication Server, a Secure Stream Server, and a Secure Proxy.
Work Assignment 1 focuses on the implementation of the Secure Stream Server and the Secure Proxy. The Secure Stream Server delivers encrypted blocks of MPEG-4 media content, while the Secure Proxy receives the encrypted stream, decrypts the blocks, and forwards the resulting media data to a web browser for playback.
The Authentication Server, which will be introduced in **Work Assignment 2**, is responsible for authenticating users operating the Secure Proxy through an authentication protocol and issuing authentication credentials. The Secure Proxy subsequently presents these credentials to the Secure Stream Server when requesting access to media content.
The implementation will be evaluated experimentally to verify its functional correctness, assess its security properties, and measure its performance.

Keywords: user authentication, Authentication Server, authentication protocols, secure streaming protocols, MPEG-4 media streaming, applied cryptography, block ciphers, stream ciphers, configurable cryptographic parameterizations

1. Introduction

In this assignment, students will design and implement 3S (Secure Streaming Service), a system for secure real-time media streaming. Students must implement and test their proposed solution, collect experimental observations, and prepare to submit the work for evaluation. They must also be prepared to explain and justify the security properties of their design with reference to the requirements in this document.

1.1 Context and Objectives

The assignment begins with a functional media-streaming system consisting of two provided components:

- **Stream Server:** Transmits a video stream over the network in media blocks generated using FFmpeg.
- **Proxy:** Receives the media blocks from the Stream Server and forwards the media data to a standard web browser for real-time playback.

The Stream Server transmits blocks containing FFmpeg-encoded media data. The Proxy requests and receives these blocks and forwards the resulting media stream to a local web browser for playback. Students should begin by studying the provided implementation, inspecting and understanding its source code, compiling the application, and verifying that it operates correctly.

The provided implementation does not employ cryptographic mechanisms and offers no security guarantees for communication between the Stream Server and the Proxy. The Stream Server transmits media data in plaintext, and neither Proxy instances nor the users operating them are authenticated. Consequently, any party capable of running a Proxy and accessing the Stream Server can retrieve and view the media stream. Furthermore, an attacker with access to the network path may be able to intercept and potentially modify the transmitted data.

The original implementation does not provide a secure-channel abstraction for communication between the Proxy and the Stream Server. As a result, it lacks protection against several classes of attacks, including man-in-the-middle attacks, peer

impersonation, network eavesdropping, replay attacks, message tampering, and message forgery. In particular, the implementation does not guarantee peer authentication, data confidentiality, message authenticity, or message integrity.

The first task is to understand the provided implementation and its operation. Students must then extend it into a secure streaming solution by designing, implementing, and testing the mechanisms and services required to provide the security properties described below.

Beyond the work assignment statement in this document, additional information and FAQ is also provided in a separated document.

1.2 Objectives and Requirements for the Work Assignment 1

The goal is to design, implement, and test a secure streaming solution based on the existing, insecure Proxy–Stream Server architecture. The solution must provide the following security properties and capabilities:

- Secure communication channel for streaming. The system must dynamically establish a secure communication channel between an authenticated Proxy and the Stream Server from which it requests media for playback. The setup must support negotiation of cryptographic algorithms, cipher suites, and associated parameters, as well as the establishment and management of cryptographic keys.

The secure channel must provide the following guarantees:

- Confidentiality: protection of media content and other sensitive information against unauthorized disclosure
- Message integrity and authenticity: message ordering guarantees with detection of unauthorized message modification and message forgery.
- Replay protection: detection and rejection of unauthorized retransmissions of previously exchanged messages.
- Message-forging protection

To meet these requirements, the following components must be designed, implemented, and tested for functional correctness and compliance with the specified security requirements:

- **Secure Proxy**: extends the original Proxy implementation with the functionality required to participate in authentication and establish secure streaming sessions.
- **Secure Stream Server**: extends the original Stream Server to support authenticated and protected streaming interactions with Secure Proxy instances.

The Secure Stream Server must be able to for streaming any Mpeg4 file that can be played by a browser, when connecting the Secure Proxy

2. Approach to the Solution

2.1 Secure Streaming Protocol Between the Secure Proxy and the Secure Stream Server

The initial interaction between the Secure Proxy and the Secure Stream Server must securely negotiate the cryptographic configuration used to establish the communication channel. Once the channel has been established, the streaming protocol must provide, for all transmitted blocks containing streamed FFMPEG segments:

- Message authentication for all transmitted media blocks, including blocks containing FFmpeg-produced media segments.
- Replay protection, including detection and rejection of replayed messages.
- Message integrity.
- Message-ordering guarantees, including detection of missing, duplicated, or out-of-order blocks where required by the protocol.
- Detection and rejection of forged or invalid messages.

The Secure Proxy and Secure Stream Server must use a secure Diffie–Hellman key-exchange protocol to negotiate and establish symmetric keys and any other parameters required for the secure channel. The design must explain how the exchange is authenticated to prevent man-in-the-middle attacks.

2.2.1 Establishing the Secure Channel

Initial configurations

To establish a secure channel for the Secure Streaming Protocol supported y the Steram Server and Proxy, the idea is to use initial configurations for both.

- For the SecureProxy, the above proposals correspond to the content of a configuration file used by the Proxy. Different configurations can be used in the configuration: file proxycrypto.config
- For the Stream Server, a configuration file is also necessary: file sservercrypto.config with a similar structure and format, as used by the proxycrypto.config

Initial negotiation handshake

By using the cryptographic configuration defined in the proxycrypto.config file, the Proxy first sends a message to the Stream Server containing an initial handshake message with the following content:

- Proposed symmetric cipher (that can use a Stream Cipher or a Block Cipher) for protecting the streaming session.
- Proposed Cipher-suite configuration

Upon receiving the message, the Stream Server must see if the proposed cryptographic configurations are compatible wth the definitions in the file sservercrypto.config. If it is valid, the Stream Server sends a confirmation message accepting the cryptographic configuration as proposed by the Proxy. Otherwise, an error message is sent to the proxy, meaning that the cryptographic proposal is not acceptable (because it is not compatible with the cryptographic configuration defined for the Stream Server). In case of acceptance, the Proxy will send a third message, also using the negotiated cryptographic configuration, as a signaling message for the server to start the stream.

The initial handshake is concluded with a third message from the Proxy to the Stream Server, that will finish the handshake and in case of success it is also used as a signaling message, in which the Proxy requests the movie to start the streaming. For the handshake processing, any violation of the correct flow or correct processing of expected messages, must be notified with an alert-error message.

2.2.2 Format for Configuration files

For this assignment different configurations can be used. The more configurations are supported in config files, the better will be the implementation.

Configuration files for the Proxy and Stream Server

```
<?xml version="1.0" encoding="UTF-8"?>
<ProxyCryptoConfig version="1.0">
      <SymCipherSuite>
        <Algorithm>ALG</Algorithm>            // Ex: AES, ChaCha20, RC6, etc
        <Mode>MODE</Mode>                     // Ex: GCM, CCB, CTR, Poly1305 or NULL
        <InitializationVector>
            <IV>IV Value</IV>                 // Ex: hexadecimal or base 64 rep. or NULL
            <IVSIZE="bits">IV Size</Length>   // Number of bits of IV or NULL
        </InitializationVector>
        <Padding>Paddingscheme</Padding>      // Ex: PKCS5Padding or NULL
      </SymCipherSuite>
      <MACsuite>
      <Algorithm>ALG<//Algorithm>.  //Ex: HMAC-SHA256, HMAC-SHA384, HMAC-SHA-512 or NULL
      </MACSuite>
      <ConfiguredKeys>
        <SymmetricKey>
             <SKey>key</SKey>          // Key material in hexadecimal or base64
             <Keysize>SIZE</Keysize>   // 128, 192 or 256 bits
        </SymmetricKey>
        <MACKey>
             <MacKey>key-mac</MacKeye> // Mac Ley Value in hexadecimal or Base64 or NULL
             <MacKeySize>Size</MacKeySize>    // Number of bits of the Mac Key, or NULL
        </MACKey>
    </ConfiguredKeys>
</ProxyCryptoConfig>
```

2.2.3 Reference formats for Handshake Messages

All the messages in the handshake will use, as reference, the following specification

Header || Payload

Header

The Header is always sent in plaintext and must have the following parts:

HS Version:Release || HS message type || Size of Payload

- HS Version and release: version and release (two bytes each) of the current handshake protocol. For Work Assignment1, version:release is 1:1 (and for successive evolutions, this can be changed, ex: 1:1, 1:2, … 2:0, 2:1, …)
- HS message type: is used as an identifier (1 byte) corresponding to different message types of the handshake. Can use the following values:
    - Type 1: first message of the handshake (sent from the Proxy to the Server)
    - Type 2: second message of the handshake (sent from the Server to the Proxy)
    - Type 3: third message of the hadshake (sent from the Proxy to the Server)
    - Type4: message with an encrypted payload (used for the secure styreaming messages, sent form Serer to the proxy
    - Type 5: message with an encrypted payload, signaling the last Block of a steram
    - Type 255: corresponds to an error-alert message

Payload:

Depending on the message types of the handshake, payloads are different

For Type1: payload contains the proposed cryptographic configuration, as a string in the following way:

SECX || PCC

- PCC: Proposed Cryptographic configuration in format:
  ALG/MODE/IV/Padding (corresponding to the configuration in the proxy config file)
  SEQX: Random number generated by the Proxy to send to the server, as a first sequence number

You must note that for possible configurable modes, IV and PADDING can be NULL

For Type 2: the payload contains an encrypted message (sent by the server to the proxy) in the following format

SEQY || SRV || CCC

- SRV - Secure Random Value: a 256 bit nonce generated and sent by the server as a challenge
- SEQY: corresponding to SEQNP + 1 (iterating the SEQNP in message type 1)
- CCC - Cryptographic configuration confirmation: a String similar to the cryptographic configuration expressed in message Type 1

For Type 3: the payload contains an encrypted message in the following format

SEQZ || R || MEDIA, where:

- SEQZ: corresponding to SEQY+1
- R: Is a response to the SRV challenge, using some heuristic (ex., an hash SHA256 (SRV)
- MEDIA: the name of the media (movie) the Proxy wants to receive as a stream

For Type 4: the payload is an encrypted message, corresponding to an encrypted block (in the stream)m with the following format:

SEQB || Encryptedblock

- SEQB; The sequence number of block, starting form SEQZ+1 and successively iterated until the last Block
- Encrypted Block: encrypted block of stream

For Type 5: the payload is an encrypted message, corresponding to an encrypted block (in the stream)m with the following format:

SEQF: Encrypted Message |

SEQF: Sequence number of the last block

Encrypted Message: Encrypted message containing ENDBLOCK

ENDBLOCK: SEQF || “END” || Random STRING

Note that the configuration file must be consistently configured with compatible definitions, and you must know how to write configurations in the right way, depending on what you want to use. Also, remember that certain cryptographic configurations don’t require padding and MACs, for example:

AES/GCM

ChaCha20/poly1305

When AES-GCM or ChaCha20-Poly1305 is used correctly, the streaming protocol can protect each media block using authenticated encryption with associated data (AEAD). In such cases, the Padidng and MAC configurations can be defined as NULL

2.2.4 Cryptographic Configurations and Secure Streaming Protocol

Students must understand how to configure the available cryptographic options and explain how the selected cryptographic constructions provide confidentiality, integrity, and authenticity. When implementing and processing these configurations, the Secure Proxy and the Secure Stream Server must correctly manage nonces, authentication tags, sequence numbers, and any required associated data, as appropriate for the selected cryptographic mechanisms.

VERY IMPORTANT FOR THE QUALITY OF IDESIGN AND MPLEMENTATION: Both the Secure Proxy and the Secure Stream Server must be fully configurable. Cryptographic parameters, algorithm selections, and security-related settings must not be hard-coded into the source code. Instead, they must be defined through an appropriate configuration mechanism, allowing the cryptographic setup to be changed without modifying or recompiling the application code.

3. Threat Model

Students must be able to justify the security guarantees provided by their design in the context of a clearly defined **threat model (adversary model)**. The report must explicitly describe the attacker's capabilities, assumptions, and limitations. At a minimum, the threat model must assume that an attacker can eavesdrop on, modify, inject, replay, reorder, or drop messages exchanged between the Secure Proxy, the Authentication Server, and the Secure Stream Server.

The report must include a table mapping each identified threat, including but not limited to the examples listed below, to the security mechanism used to mitigate it. If a threat is not addressed by the proposed solution, the report must explicitly state that it falls outside the scope of the implementation.

The security analysis must address, at a minimum, the following threats and security requirements:

- **Man-in-the-middle attacks:** Attacks against the key-agreement protocol used to establish a secure channel between the Secure Proxy and the Secure Stream Server, including the mechanisms required to authenticate the communicating parties and protect the Secure Streaming Protocol.
- **Attacks on media-stream blocks:** Eavesdropping, tampering, replay, and reordering of media-stream blocks transmitted through the Secure Streaming Protocol.
- **Session-key compromise and forward secrecy:** The consequences of session-key compromise and whether the key-establishment mechanism provides forward secrecy across successive streaming sessions.

For this project, students may assume that endpoint systems are not compromised. Specifically, the Secure Proxy and the Secure Stream Server, each running on its respective host, are assumed to be free from intrusion and under the control of legitimate operators. Endpoint compromise is therefore outside the scope of the threat model.

Furthermore, students may assume that the underlying cryptographic primitives remain secure when correctly implemented and configured. The threat model does not include attacks that break these primitives directly. However, students must consider implementation errors, incorrect cryptographic configurations, and improper protocol usage when evaluating the security of their solutions.

4. Validation Tests and Experimental Observations

As an initial functional test, students must verify that a requested video can be streamed from the Secure Stream Server, processed by the Secure Proxy, and played in a web browser in real time. A preliminary assessment may be performed through visual inspection of the playback. However, a more rigorous evaluation requires the Secure Proxy to collect and report instrumentation metrics after receiving and processing the complete media stream.

At a minimum, the Secure Proxy must report the following statistics:

- **Number of encrypted blocks received:** The total number of encrypted media blocks received from the Secure Stream Server.
- **Number of decrypted blocks forwarded:** The total number of successfully decrypted media blocks forwarded to the browser for playback.
- **Observed stream throughput:** The measured throughput, expressed in kilobytes per second (kB/s) or megabytes per second (MB/s), with the measurement methodology clearly specified.
- **Total volume of processed data:** The total size of the media blocks processed by the Proxy, together with a comparison against the size of the original media file transmitted by the Secure Stream Server.

Students should explain the collected measurements and discuss any discrepancies between the original media-file size and the volume of data processed by the Proxy.

5. Suggested Development Plan

The following steps provide an indicative development plan rather than a mandatory sequence. After completing the initial code analysis (Step 0), students may define their own development methodology, reorganize the steps, and carry out tasks in a different order, provided that all required objectives are achieved and adequately validated.

Students working in pairs are encouraged to adopt a collaborative and incremental development approach, distributing tasks and performing activities in parallel whenever feasible. Integration and testing should be conducted regularly to ensure that independently developed components operate correctly together.

- **Step 0 — Initial Code Analysis:** The initial implementation, consisting of a Proxy, a Stream Server, and a Streaming Protocol, is provided. Students must analyze and understand its architecture and operation by inspecting the source code, compiling the application, and executing the provided implementation.
- **Step 1 — Secure Proxy Design and Implementation:** Design and implement the Secure Proxy to support the Secure Streaming Protocol and configurable cryptographic parameters specified in the proxycrypto.config configuration file. The implementation must support secure communication with the Secure Stream Server without requiring cryptographic parameters to be hard-coded into the source code.
- **Step 2 — Secure Stream Server Design and Implementation:** Design and implement the Secure Stream Server to support the Secure Streaming Protocol and configurable cryptographic parameters specified in the sscrypto.config configuration file. The implementation must interoperate correctly with the Secure Proxy and support encrypted media streaming. This step is worth **6 points** for a correct and complete implementation, including the required functionality and testing with at least one cryptographic configuration.
- **Step 3 — Validation with a Single Cryptographic Configuration:** Validate the correctness, completeness, and integrated operation of the Secure Proxy, Secure Stream Server, and Secure Streaming Protocol using at least one consistent cryptographic configuration. Verify successful secure-channel establishment, encrypted media transmission, correct decryption, and playback in a web browser.
- **Step 4 — Validation with Multiple Cryptographic Configurations:** Validate the correctness and interoperability of the Secure Proxy, Secure Stream Server, and Secure Streaming Protocol using different cryptographic configurations, including the combinations specified in the assignment.

  Students may be required to demonstrate the implementation using different combinations of cryptographic parameters. They must therefore understand how to configure both components consistently for each combination. The implementation must support configuration changes without requiring recompilation or modification of the source code. Cryptographic algorithms and parameters must be selected through the appropriate configuration files, subject to the combinations supported by the implementation.
- **Step 5 — Optimization and Extensions:** Optimize the implementation and consider additional features. Possible extensions include:

- **Concurrency and parallelism:** Support and test multiple concurrent Proxy instances requesting different movies from the Stream Server. Verify that concurrent streaming sessions operate correctly and independently.
- **Additional extensions:** Propose and implement other improvements that enhance the system's functionality, performance, scalability, or security. Such extensions may be considered for additional recognition in the evaluation, subject to the assessment criteria.

6. Indicative Evaluation Criteria

The following criteria provide an indicative framework for assessing the correctness, completeness, and quality of the implementation. The implementation component accounts for **75% of the overall assignment grade**.

The implementation is evaluated on a 20-point scale, according to the following criteria:

- **Step 1: Secure Proxy:** 4/20 points. Assessment of design correctness, implementation correctness, completeness, and compliance with the specified requirements.
- **Step 2: Secure Stream Server:** 4/20 points. Assessment of designed correctness, implementation correctness, completeness, and compliance with the specified requirements.
- **Step 3: Integrated Validation with One Cryptographic Configuration:** 4/20 points. Assessment of the correct and complete operation of the Secure Proxy, Secure Stream Server, and Secure Streaming Protocol using at least one consistent cryptographic configuration.
- **Step 4: Validation with Multiple Cryptographic Configurations:** 4/20 points. Assessment of interoperability, correctness, and the ability to switch between supported cryptographic configurations without recompilation.
- **Step 5: Optimization and Extensions:** 4/20 points. Assessment of performance improvements, support for concurrent streaming sessions, and other relevant extensions.